

MTRouter: 使用历史—模型联合嵌入的成本感知多轮 LLM 路由

Yiqun Zhang^{1,2} Hao Li² Zihan Wang¹ Shi Feng^{1,†} Xiaocui Yang¹

Daling Wang¹ Bo Zhang² Lei Bai² Shuyue Hu^{2,†}

¹ 东北大学计算机科学与工程学院

中国沈阳 110819

² 上海人工智能实验室

yiqunzhang@stumail.neu.edu.cn

{fengshi,yangxiaocui,wangdaling}@cse.neu.edu.cn

{lhao4,zhangbo,bailei,hushuyue}@sjlab.org.cn

摘要

多轮、长时程任务正越来越常见于大语言模型 (LLM) 的应用中, 但解决这类任务通常需要大量顺序模型调用, 因而累积可观的推理成本。本文研究成本感知的多轮 LLM 路由: 在固定成本预算下, 每一轮从模型池中选择要调用的模型。我们提出 MTRouter: 它将交互历史与候选模型编码为联合的历史—模型嵌入, 并从已记录的轨迹中学习结果估计器, 以预测轮级模型效用。实验表明, MTRouter 改善了性能—成本权衡: 在 ScienceWorld 上, 它超过 GPT-5, 同时将总成本降低 58.7%; 在 Humanity’s Last Exam (HLE) 上, 它以相对 GPT-5 降低 43.4% 总成本的代价取得有竞争力的准确率, 并且这些收益还能迁移到留出任务。进一步分析揭示了其有效性的若干机制: 相较先前的多轮路由器, MTRouter 更少切换模型、更能容忍瞬时错误, 并在模型间展现出涌现的专长分工。代码见 <https://github.com/ZhangYiqun018/MTRouter>。

1 引言

大语言模型 (LLM) 正日益被部署来解决需要长交互序列的复杂工具使用任务, 例如软件工程 (Jimenez et al., 2023; Jain et al., 2024) 和多步推理 (Wang et al., 2022; Yang et al., 2024; Team et al., 2025b; Phan and Alice Gatti, 2025)。这类 LLM 的一个决定性特征是其长时程性: 每个 episode 通常需要几十次顺序模型调用。随着轨迹变长, 累积推理成本—又受到上下文窗口扩大 (Dao, 2023) 和 token

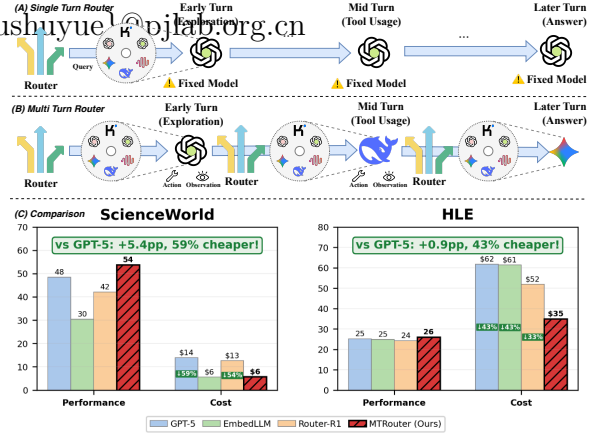


图 1: 上: 单轮 (episode 级) 路由器选择一个模型, 并在整个 episode 中保持不变。中: 多轮路由器可随交互状态变化, 在不同轮次调整模型选择。下: MTRouter 在 ScienceWorld (Wang et al., 2022) 与 Humanity’s Last Exam (HLE) (Phan and Alice Gatti, 2025) 上, 比有代表性的基线取得更优的性能—成本权衡。

消耗超线性增长 (Gao and Peng, 2025) 的加剧—成为实际部署与可复现研究的主要障碍。

当前模型生态中存在显著的经济差距, 这进一步加剧了这一压力: 前沿模型提供最先进的推理能力 (如 claude-opus-4.5¹、gpt-5²), 但成本通常比轻量模型高两个数量级 (如 DeepSeek-v3.2 (Liu et al., 2025)、Kimi-K2 (Team et al., 2025a))。为降低成本, 已有工作探索了单轮 (episode 级) 路由: 根据初始提示词在 episode 开始时选择一个模型 (图 1, 上)。但这种静态方法对于长时程任务天然次优。一条 episode 通常包含异质步骤, 从高风险的战略规划到例行工具调用和数据格式化。每一轮都使用前沿

[†] 通讯作者。

¹ <https://www.anthropic.com/claude/opus>

² <https://openai.com/index/gpt-5-system-card>

模型会造成浪费，而只依赖廉价模型则可能在关键节点引发灾难性失败。

这一观察促成了多轮（轮级）路由：智能体在交互的每一步自适应地切换模型（图 1，中）。尽管有前景，多轮路由面临显著的预测挑战：路由器必须评估当前轮的一个特定模型选择，是否会在数十步之后危及最终结果。简单启发式规则或局部错误检测通常无法捕获模型表现对整条轨迹的长期影响。

我们提出 **MTRouter**，它从已记录轨迹中学习一个结果估计器。该估计器将历史-模型对映射为最终 episode 结果（终止得分/准确率）的估计，并通过考虑错误的调整从离线数据中获得稳定的训练信号。推理时，路由器在每一轮选择预测结果最高的候选模型，并在固定成本和轮数限制下评估 episode。

在实证上，MTRouter 在性能和成本两方面均带来一致收益。在 ScienceWorld（测试集）上，它将平均得分从 48.4 (GPT-5) 提升至 53.8，同时将总成本降低 58.7%；在 HLE（测试集）上，它以降低 43.4% 总成本实现有竞争力的准确率。它还可于语义分布偏移（OOD）下泛化，以显著成本节省保持更好的结果（表 3）。除汇总指标外，我们的分析还表明，多轮路由并不等于“更多切换”：MTRouter 比 Router-R1 (Zhang et al., 2025a) 以更少模型切换获得成功，对瞬时错误更具容忍性，并呈现结构化模型使用与跨工具/动作的涌现专长（图 3-6）。

我们的主要贡献如下：

- 我们提出 **MTRouter**：它从离线轨迹中学习历史-模型对上的结果估计器，并在多轮智能体 episode 中执行轮级路由。
- 我们在 ScienceWorld 和 HLE(测试与 OOD) 上以固定候选池开展评估，结果表明，相较于 Router-R1 与有代表性的商用路由器在内的强基线，MTRouter 在性能和总成本上均有一致改善。
- 我们提供分析，将这些收益关联到具体路由行为，包括更少不必要的切换、更好的错误恢复，以及跨工具/动作的涌现专长。

2 相关工作

LLM 路由。 具有不同成本-能力特征的 LLM 大量涌现，推动了智能模型选择的研究。FrugalGPT (Chen et al., 2023) 将模型从廉价到昂贵依次级联，直至达到置信度阈值。EmbedLLM (Wang et al., 2024) 学习嵌入，以预测模型在特定查询上的表现。RouterDC (Chen et al., 2024) 使用双重对比学习训练有效的路由器。Avengers (Zhang et al., 2025c) 使用简单的聚类式路由方案编排十个 7B 模型池，实现超过 GPT-4.1 的性能；AvengersPro (Zhang et al., 2025b) 则将这一设计扩展为在平衡评测设置下提供帕累托最优的性能-成本权衡。这些方法聚焦于单轮路由：给定一个查询，选择一个模型作答。面向多轮场景，Router-R1 (Zhang et al., 2025a) 训练策略 LLM，通过强化学习交错进行推理和路由。不同于依赖重型 LLM 路由器的 Router-R1，我们的方法在联合历史-模型嵌入上学习轻量级结果估计器，实现高成本效益的轮级路由。

LLM 工具使用。 由工具增强的 LLM 能够完成复杂的多步任务 (Schick et al., 2023; Qin et al., 2023)。ReAct (Yao et al., 2022) 在单个提示词内交错推理与动作；Toolformer (Schick et al., 2023) 则微调模型以自主调用工具。近期研究已在多种领域探索基于 LLM 的工具使用，包括软件工程 (Jimenez et al., 2023; Yang et al., 2024)、网页浏览 (Zhou et al., 2023)、操作系统交互 (Xie et al., 2024)、复杂推理 (Mialon et al., 2023) 与自主研究 (Team et al., 2025b)。这些任务本质上具有交互性，通常无法在单轮内解决，因而需要重复的动作-观察循环。多数工具使用框架依赖固定的底层模型；我们的工作引入一个在模型间动态切换的路由层，将模型选择视为每一步的自适应决策。这一方法与现有工具使用方法互补，并可推广至多轮 LLM 系统。

3 MTRouter

3.1 问题形式化

单轮 (episode 级) 路由在 episode 开始时作出一次模型选择, 并始终保持不变。我们研究多轮模型路由, 其中模型选择可随时间变化。一条 episode 由以 t 为索引的轮次组成。在每一轮, 路由器选择模型 $a_t \in \mathcal{A}$ 来生成智能体的下一个输出 (路由器本身也可以由 LLM 实现)。一个轮次是一轮交互, 且恰好包含一次对所选模型的调用: 给定历史 h_t (任务描述、对话上下文和最新观察), 所选模型产生 $y_t \sim p_{a_t}(\cdot | h_t)$, 解析器将 y_t 映射为可执行动作 $u_t = \text{parse}(y_t)$; 执行 u_t 得到下一条观察 o_{t+1} 。

当任务完成 (由环境或智能体提交指示)、达到轮数上限, 或成本预算耗尽时, episode 结束。环境提供终止得分 S_{final} 。

目标。 我们在每条 episode 的成本预算下优化任务性能:

$$\max \mathbb{E}[S_{\text{final}}] \quad \text{s.t.} \quad \sum_{t=0}^{T-1} c_t \leq B \quad (1)$$

其中 c_t 是第 t 轮成本 (由 token 使用量和模型定价计算), B 是每条 episode 的成本预算, $T \leq T_{\text{max}}$ 是 episode 长度 (受最大轮数上限约束)。当预算耗尽或达到轮数上限时, episode 终止。

3.2 联合历史-模型表征

路由决策既取决于交互历史, 也取决于所选模型 (即二元组 (h_t, a_t))。因此, 我们分别嵌入历史和候选模型, 再将它们组合为路由器使用的联合表征。

历史编码。 我们将路由历史表示为任务块加上累积交互上下文。具体而言, 使用固定模板序列化每个示例:

$$h_t = [q, \langle u_0, o_1 \rangle, \dots, \langle u_{t-1}, o_t \rangle] \quad (2)$$

实现时, 我们不保留固定数量的历史轮次 K 。相反, 我们采用最大长度 8192 token 的 token

预算截断: 始终保留任务块, 在剩余预算内保留最新交互上下文, 并首先截断最早上下文。随后, 冻结的文本编码器 ϕ 对序列化历史编码, 得到 $z_x = \phi(h_t) \in \mathbb{R}^d$ 。

模型编码。 每个候选模型 $a \in \mathcal{A}$ 由可学习嵌入 $z_a = \psi(a)$ 表示; 它结合 (i) 结构化属性向量 attr_a (包括上下文长度上限、知识截止日期和定价), 以及 (ii) 捕获元数据无法解释的模型特定行为的可学习残差 e_a 。最终模型嵌入通过线性投影拼接这些成分:

$$z_a = W_{\text{proj}} \cdot [\text{MLP}(\text{attr}_a); e_a] + b_{\text{proj}} \quad (3)$$

其中 e_a 是经正则化以防止过拟合的残差嵌入。

联合表征。 我们通过拼接两个嵌入构成联合特征向量 $[z_x; z_a]$, 并将其输入共享前馈骨干网络, 以支持模型条件化预测。为高效地对多个候选项打分, 我们只对当前历史计算一次 z_x , 再将其批量地与每个候选模型嵌入拼接。

3.3 学习结果估计器

我们学习结果估计器 $\hat{s}_\theta(h_t, a)$, 将历史-模型对 (h_t, a) 映射为在第 t 轮选择模型 a 时的期望终止结果。我们将 $\hat{s}_\theta$ 参数化为轻量前馈网络 (采用 ReLU 非线性和可选 dropout 的 MLP); 它接收联合表征 $[z_x; z_a]$ 并输出单个标量:

$$\hat{s}_\theta(h_t, a) = f_\theta([z_x; z_a]) \in \mathbb{R}. \quad (4)$$

我们用终止结果而非稠密的逐轮奖励监督该估计器, 原因有二: (i) 在复杂智能体环境中, 忠实的中间奖励通常不可得; (ii) 为稠密反馈训练稳定的奖励模型可能很脆弱。在我们的设置中, episode 同时受成本预算 B 和最大轮数 T_{max} 约束, 因此昂贵选择和浪费轮次 (例如触发重试的错误) 已反映在记录的终止得分中; 我们因而不为目标额外加入单独的成本惩罚。

带错误惩罚的结果目标。 对于已记录轨迹的每一轮, 我们通过以累计错误惩罚调整终止得分 S_{final} , 构成一个轮次条件化目标。具体而言, 我们从轨迹日志中检测错误 (例如无效动

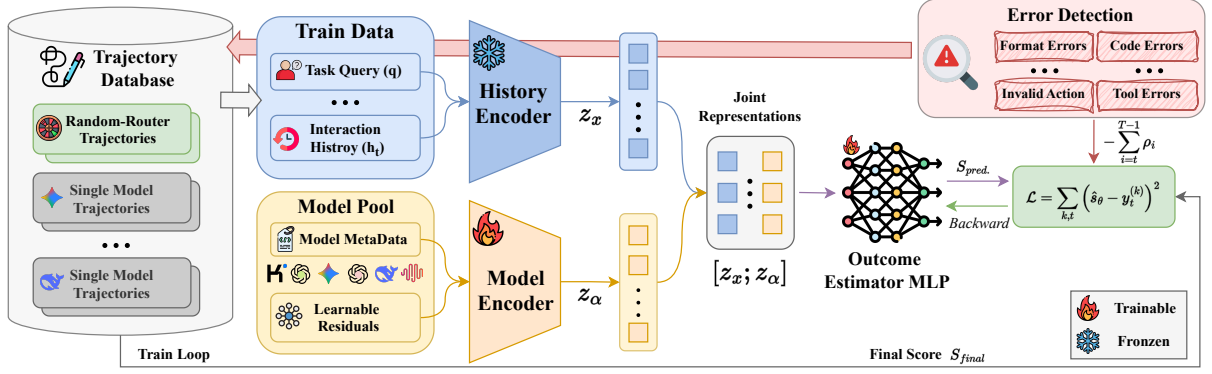


图 2: MTRouter 概览。

作、解析/格式违规)，并按严重程度和轮次进度进行惩罚：

$$\tilde{S}_t = S_{\text{final}} - \sum_{i=t}^{T-1} \rho_i, \quad (5)$$

$$\rho_i = \mathbf{1}[\mathcal{E}_i \neq \emptyset] \cdot \beta_{\text{sev}(\mathcal{E}_i)} \cdot w\left(\frac{i+1}{T_{\text{max}}}\right), \quad (6)$$

其中 $\mathcal{E}_i$ 是第 i 轮检测到的错误类型集合， $\text{sev}(\mathcal{E}_i)$ 取该轮错误中的最高严重程度， β 是相应的严重程度系数。进度权重 $w(\cdot)$ 单调递增（我们使用简单的分段线性日程），因此后期错误受到更强惩罚。这体现了一个简单的设计直觉：智能体仍在收集信息时，早期错误可能可恢复；而后期错误会更直接地危及任务完成，因此我们对接近 episode 末尾的错误施加更低容忍度。我们训练 $\hat{s}_\theta(h_t, a)$ 近似 $\mathbb{E}[\tilde{S}_t | h_t, a]$ 。

训练目标。 我们在由随机路由器收集的离线轨迹上训练。对于轨迹 k 中的每一步 $(h_t^{(k)}, a_t^{(k)})$ ，使用监督目标 $y_t^{(k)} = \tilde{S}_t^{(k)}$ ，并最小化平方误差损失：

$$\mathcal{L} = \sum_{k,t} \left(\hat{s}_\theta(h_t^{(k)}, a_t^{(k)}) - y_t^{(k)} \right)^2 \quad (7)$$

数据收集。 我们将每条 episode 记录为一条轨迹（一系列轮级元组），并从两个来源收集训练轨迹：(i) 在每一轮从模型池采样模型的**均匀（随机）路由器**；(ii) 在训练任务上由一个候选模型处理整条 episode 的**单模型**运行。随机路由器数据提供模型选择的广泛覆盖，单模型数据则以一致的逐模型行为锚定估计器。在两个基准上，离线训练集包含 1,291 个训练实

例、29,693 条轨迹和 515,221 个总轮次，估计一次性收集成本约为 \$1,620。

推理。 部署时，路由器选择预测结果最高的模型：

$$a_t^* = \arg \max_{a \in \mathcal{A}} \hat{s}_\theta(h_t, a), \quad (8)$$

并在每条 episode 的预算 B 和轮数限制 T_{max} 下执行（任一限制达到时终止）。

4 实验

我们在两个具有挑战性的多轮基准上评测 MTRouter，并分析学习到的路由模式。除非另有说明，每个实验均重复三次并报告均值。匿名代码仓库见 <https://github.com/ZhangYiqun018/MTRouter>。

4.1 实验设置

基准。 我们在两个多轮基准上评测：**ScienceWorld** (Wang et al., 2022) 和 **HLE (Humanity’s Last Exam)** (Phan and Alice Gatti, 2025)。ScienceWorld 是一个要求程序化科学推理的文本交互环境，其终止得分为 $S_{\text{final}} \in [-100, 100]$ 。HLE 是跨学术领域的长上下文基准，问题需要借助工具进行多步推理，成功为二元结果。我们同时考察分布内 (ID) 和分布外 (OOD) 切分。我们并非依靠随机重采样，而是通过为 OOD 留出完整任务类型/学科类别，构建与训练和 ID 测试语义不相交（无重叠）的评测。对于 ScienceWorld，我们使用 13 个任务类型作为 ID，并留出 12 个任务类型作为 OOD；

基准	错误类别	规则数
HLE	格式错误	4
	Python 执行错误	11
	搜索工具错误	3
	浏览工具错误	5
ScienceWorld	无效动作	1

表 1: 用于错误检测的错误类别。HLE 具有多种工具相关错误, 而 ScienceWorld 只惩罚无法解析的动作。完整规则规范见附录 B。

完整任务类型切分见表 5。在 ID 任务类型内, 我们进一步将任务变体以 60%/20%/20% 切分为训练/验证/测试 (附录 A)。对于 HLE, 我们使用 6 个学科类别作为 ID, 并留出 2 个类别作为 OOD; 详细类别切分见表 6。我们通过将基准的测试问题划分到选定的学科组来报告 ID 与 OOD 性能 (表 6)。

工具配置。 对于 HLE, 我们遵循 TongYi-DeepResearch (Team et al., 2025b) 的工具使用设置, 启用四种工具: search、browse、python 和 answer。search 使用 Serper 的 Google Search API; browse 获取网页内容 (启用时通过 Jina Reader), 并可选择性地摘要长网页以控制上下文长度 (附录 E.5); python 支持确定性计算, answer 用于提交最终响应。ScienceWorld 不需要外部网页工具; 智能体每一轮通过单条文本动作命令与模拟器交互, 并使用内置查询命令 (如 ?navigation、?object) 枚举有效动作。工具 schema 在运行时注入 HLE 系统提示词, 列于附录 E.3。

错误检测。 我们从环境观察中检测错误, 以计算训练时退火错误成本。表 1 按基准汇总错误类别。HLE 错误涵盖格式违规、Python 执行失败和工具特定问题; ScienceWorld 只惩罚无法解析的动作 (“door is not open” 之类的环境反馈属于正常探索, 而非错误)。严重程度 (高/中/低) 决定 AEC 计算中的惩罚系数。完整规则规范见附录 B。除非另有说明, 全文使用高 =1.0、中 =0.8、低 =0.2 的严重程度系数。

模型池。 我们使用跨越 $20\times$ 成本区间的 6 个前沿 LLM 进行评测 (见表 2)。

模型	上下文	输入 \$/M	输出 \$/M
GPT-5 (OpenAI, 2025)	400K	1.25	10.00
DeepSeek-V3.2 (Liu et al., 2025)	164K	0.27	0.42
MiniMax-M2 (MiniMax, 2025)	197K	0.20	1.00
Kimi-K2 (Team et al., 2025a)	131K	0.39	1.90
Gemini-2.5-Flash-Lite (Kilpatrick and Gleicher, 2025)	1M	0.10	0.40
GPT-OSS-120B (OpenAI et al., 2025)	131K	0.09	0.36

表 2: 带定价的模型池 (来自 OpenRouter)。

实现细节。 我们使用 AdamW 优化器 (学习率 10^{-3} 、权重衰减 0.01) 和余弦退火训练 100 个 epoch, 并采用早停 (patience=3)。批量大小为 64。我们使用冻结的 Qwen/Qwen3-Embedding-0.6B 编码器对历史编码, 它产生 1024 维嵌入, 并将最大输入长度设为 8192 token。模型编码器将 8 维元数据特征向量映射为 32 维属性嵌入, 与 16 维逐模型残差嵌入拼接后, 通过线性投影得到 64 维模型嵌入。为防止过早收敛, 我们对可学习残差嵌入施加 L_2 惩罚 ($\lambda = 0.001$)。对于式 5 中的错误惩罚, 我们使用 $p_0=0.3$ 、 $p_1=0.7$ 、 $w_{\min}=0.3$ 和 $w_{\max}=1.0$ 的分段线性 warmup 实例化进度权重 $w(\cdot)$ (附录 B)。评测时, ScienceWorld 的最大时程为 50 步, HLE 为 30 步, 两个基准的每条 episode 成本上限均为 \$2.0。

基线。 我们比较三组基线:

- **单模型:** 模型池中的每个候选模型独占处理完整 episode。
- **单轮路由器 (episode 级) :** RouterDC (Chen et al., 2024)、EmbedLLM (Wang et al., 2024) 和 AvengersPro (Zhang et al., 2025c,b)。这些路由器根据初始查询上下文在 episode 开始时作出一次路由决策, 再在其余轮次保持所选模型不变。
- **多轮路由器 (轮级) :** 随机路由器 (每轮均匀随机选择)、Router-R1 (Zhang et al., 2025a) (按 Router-R1 方案训练 Qwen2.5-7B-Instruct 路由模型)、LLM 路由器 (与 Router-R1 使用相同提示词, 但直接以 DeepSeek-V3.2 作为路由模型、不训练), 以

方法	ScienceWorld				HLE			
	测试		OOD		测试		OOD	
	得分 $\uparrow$	总成本 (\$) $\downarrow$	得分 $\uparrow$	总成本 (\$) $\downarrow$	准确率 $\uparrow$	总成本 (\$) $\downarrow$	准确率 $\uparrow$	总成本 (\$) $\downarrow$
单模型基线								
GPT-5	48.4 $\pm$ 2.1	13.9	4.9 $\pm$ 2.9	47.6	25.1 $\pm$ 1.6	61.8	34.8 $\pm$ 2.2	65.3
DeepSeek-V3.2	13.1 $\pm$ 2.0	2.9	-4.2 $\pm$ 2.4	22.8	15.6 $\pm$ 1.3	22.4	28.7 $\pm$ 1.6	22.2
MiniMax-M2	-0.5 $\pm$ 1.6	3.2	0.9 $\pm$ 2.3	10.9	7.8 $\pm$ 1.4	18.1	8.9 $\pm$ 1.9	9.8
Kimi-K2	5.2 $\pm$ 1.8	2.5	0.2 $\pm$ 2.1	8.9	11.4 $\pm$ 1.2	12.0	20.1 $\pm$ 1.5	9.8
Gemini-2.5-Flash-Lite	4.2 $\pm$ 1.7	0.3	-2.1 $\pm$ 2.2	1.5	5.6 $\pm$ 1.1	3.0	8.4 $\pm$ 1.5	2.1
GPT-OSS-120B	26.6 $\pm$ 2.3	0.5	1.1 $\pm$ 2.7	4.2	9.7 $\pm$ 1.0	0.7	11.4 $\pm$ 1.2	2.1
单轮路由器 (episode 级)								
RouterDC (Chen et al., 2024)	23.1 $\pm$ 2.4	3.3	5.5 $\pm$ 3.1	2.5	12.8 $\pm$ 1.6	10.3	17.9 $\pm$ 2.1	13.5
EmbedLLM (Wang et al., 2024)	30.4 $\pm$ 2.8	5.6	5.0 $\pm$ 3.4	3.0	24.8 $\pm$ 2.0	61.4	33.6 $\pm$ 2.5	56.8
AvengersPro (Zhang et al., 2025c)	36.8 $\pm$ 2.5	4.1	2.4 $\pm$ 3.2	4.1	23.7 $\pm$ 1.8	47.5	30.6 $\pm$ 2.3	33.3
多轮路由器 (轮级)								
随机路由器	21.7 $\pm$ 4.5	3.9	-8.1 $\pm$ 5.4	20.3	20.0 $\pm$ 2.8	16.9	23.8 $\pm$ 3.4	14.7
LLM 路由器	19.8 $\pm$ 3.8	12.2	-0.4 $\pm$ 4.6	28.3	24.0 $\pm$ 2.3	56.2	36.0 $\pm$ 2.8	35.6
Router-R1 (Zhang et al., 2025a)	42.1 $\pm$ 3.6	12.6	2.1 $\pm$ 4.2	21.0	24.2 $\pm$ 2.2	51.9	35.1 $\pm$ 2.6	60.7
OpenRouter [†] (OpenRouter, 2025)	-26.4 $\pm$ 3.2	3.0	-26.9 $\pm$ 3.8	15.5	18.3 $\pm$ 2.1	138.5	34.0 $\pm$ 2.5	154.3
MTRouter (本文)	53.8$\pm$3.2	5.7	9.9$\pm$3.9	16.3	26.0$\pm$2.3	35.0	38.6$\pm$3.0	31.2
Δ 相对 GPT-5	+5.4	节省 58.7%	+5.0	节省 65.8%	+0.9	节省 43.4%	+3.8	节省 52.3%
Δ 相对 Router-R1	+11.7	节省 54.4%	+7.8	节省 22.4%	+1.8	节省 32.7%	+3.5	节省 48.7%

表 3: ScienceWorld 和 HLE 基准上的主结果（测试与 OOD 切分）。性能指标报告三次运行的均值 $\pm$ 标准差；总成本为全部评测 episode 的总和。OOD 评测使用留出的任务类型（ScienceWorld）和留出的学科类别（HLE）。 Δ 行展示相对 GPT-5 和 Router-R1 的得分增益与相对成本节省。[†]OpenRouter 使用具有不同模型池的固定提供商侧路由 API（附录 D）。

及 OpenRouter (OpenRouter, 2025)（经由 OpenRouter 自动路由 API 的代表性商用路由器）。纳入 OpenRouter 是为了将 MTRouter 与现成的生产路由系统置于同一语境；但其路由 API 不允许我们定制候选模型池，且它从远大于固定 6 模型设置的池中选择（附录 D）。

4.2 MTRouter 有效吗？

表 3 展示 MTRouter 与其他基线在分布内（测试）和分布外（OOD）上的结果。

测试集。 在 ScienceWorld 上，MTRouter 获得最高平均得分 (53.8)，同时相对 GPT-5 将总成本降低 58.7%；相较 Router-R1，它得分提高 +11.7 点，且总成本低 54.4%。每条 episode 固定一个模型的 episode 级（单轮）路由器始终落后，支持了如下必要性：在阶段与错误会随时间演化的交互设置中，需要多轮路由。值得注意的是，OpenRouter 在 ScienceWorld 上

产生负分，因为它低估任务难度，过度依赖轻量模型（附录 D）。在 HLE 上，MTRouter 取得最高准确率 (26.0%)，并保持成本效率（相对 GPT-5 总成本低 43.4%，相对 Router-R1 低 32.7%）；Router-R1 和 LLM 路由器达到相近准确率，但成本更高。总体而言，MTRouter 在两个基准上均比强单模型基线和既有路由基线获得更优的准确率–成本权衡。

变体	SW 平均得分 $\uparrow$	HLE 准确率 (%) $\uparrow$
MTRouter (完整设置)	53.8 $\pm$ 3.2	26.0 $\pm$ 2.3
以 Ridge 替代 MLP	49.1 $\pm$ 3.5	23.4 $\pm$ 2.2
无随机路由器数据	47.2 $\pm$ 4.1	22.6 $\pm$ 2.1
无错误惩罚	48.5 $\pm$ 3.6	23.8 $\pm$ 2.1
无路由历史 *	44.6 $\pm$ 3.8	21.3 $\pm$ 2.0
硬编码模型编码器	41.3 $\pm$ 4.4	19.7 $\pm$ 2.1

表 4: ScienceWorld 和 HLE 测试集上的消融研究。报告带 95% 置信区间的性能。* 该消融移除路由器历史：路由仅以当前轮为条件（所选模型仍接收完整对话历史）。

OOD。 接下来考察表 3 的 OOD 列，它评测语义分布偏移（留出的 ScienceWorld 任务类型和 HLE 学科类别）。在 ScienceWorld OOD 上，MTRouter 比 GPT-5 高 +5.0 点，同时总成本低 65.8%；在 HLE OOD 上，它达到 38.57% 准确率，且总成本比 GPT-5 低 52.3%。这些结果表明，MTRouter 不仅能在分布偏移下泛化，也能保持成本效率，并在所比较方法中建立最强的整体表现。

消融研究。 表 4 汇总 ScienceWorld 与 HLE 上的消融。当以更简单的回归器替换 MLP、移除随机路由器数据或移除错误惩罚调整时，性能均下降，表明每个组件都有助于从离线轨迹中学习可靠的路由偏好。最大的性能下降来自移除路由历史，或以硬编码特征替换学习到的模型编码器，突出了同时建模演化的交互上下文和模型行为的重要性。我们使用 Ridge 作为标准的正则化线性基线，以检验性能收益是否来自非线性建模，而非仅来自特征构造。关于预算敏感性、候选池大小和历史 token 预算的额外稳健性检查见附录 C。

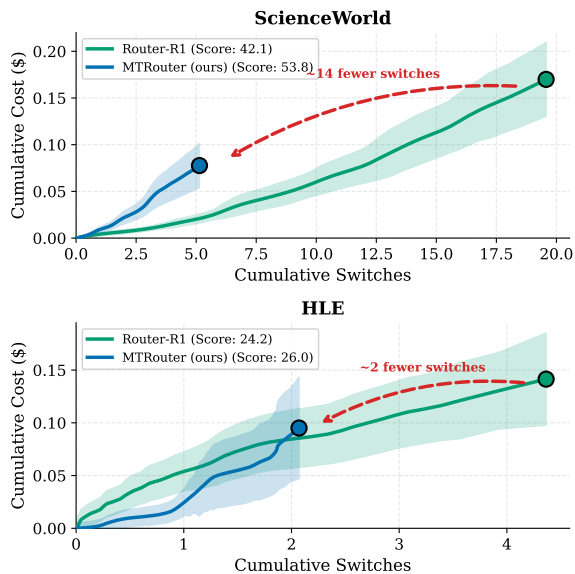


图 3: 成功 episode 中的累计成本与累计模型切换次数: 比较 ScienceWorld 和 HLE 上的 MTRouter 与 Router-R1（通过重放记录的轨迹构建）。

4.3 MTRouter 为什么有效？

尽管分布内和分布外结果（表 3）以及消融（表 4）证明了 MTRouter 的有效性，接下来我们提出一个更具诊断性的问题：它为何有效？我们采用一系列互补分析，将性能收益与具体的路由行为联系起来。

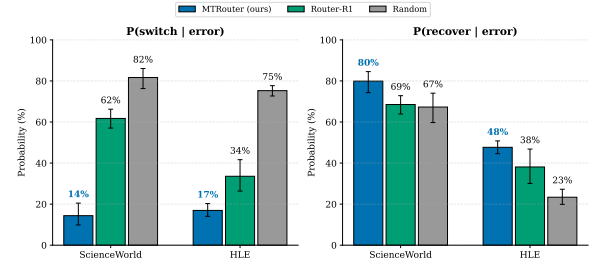


图 4: 错误触发的切换与恢复。**左：**错误（格式错误/无效动作）后切换模型的概率。**右：**以错误为条件时，下一轮恢复（无错误）的概率。

从简单诊断开始：切换与成本。 若多轮路由“只是更频繁地切换”，那么频繁切换的路由器应当占优。我们发现事实相反。图 3 在成功 episode 上绘制：随着路由器进行额外模型切换，累计 API 成本如何增长。每条曲线均由重放 MTRouter 和 Router-R1 的记录轨迹构建，沿 episode 累积逐轮成本并计数切换。尽管 MTRouter 获得更好的最终性能（表 3），其轨迹通常以更少切换和更低累计成本取得成功（例如 ScienceWorld 上：MTRouter 约 5 次切换，Router-R1 约 20 次）。

除逐 token 定价差异外，多轮设置中的频繁切换也会降低提示词缓存的有效性，降低缓存命中率并提高长历史服务的有效成本 (Hu et al., 2025)。这立即引出一个更具体的问题：Router-R1 何时选择切换，这些切换是否确有帮助？

通过更能容忍错误来减少切换。 图 4 显示，Router-R1 在错误后激进切换，而 MTRouter 的反应性较低：它常保持当前模型并尝试继续。关键在于，这并非“忽略”错误：右图显示 MTRouter 在下一轮恢复的概率更高。与之相符，发生错误后，MTRouter 在 ScienceWorld

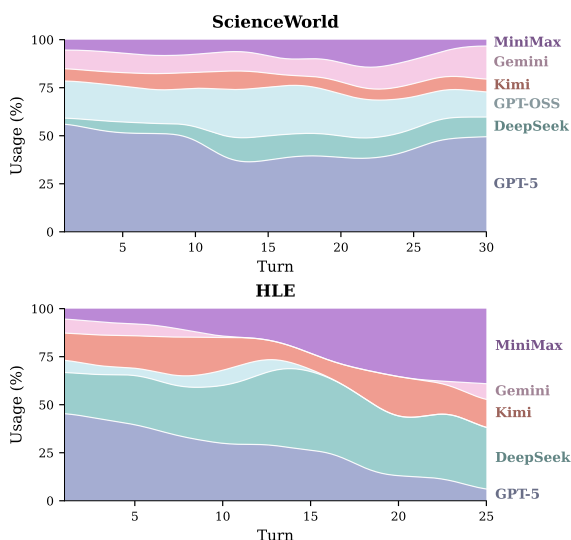


图 5: ScienceWorld 和 HLE 上按轮次划分的模型使用。MTRouter 呈现结构化、基准特定的路由行为，而不是在各轮均匀切换模型。在 HLE 中，GPT-OSS-120B 主要用于早期轮次，随后在后期衰减至近零，因此其带状区域在视觉上不易察觉。

上约 90.2% 的时间、在 HLE 上约 80.9% 的时间保持同一模型，显著高于 Router-R1（分别为 38.3% 和 66.4%）。结合图 3，这些趋势表明 MTRouter 避开了很大一部分看似低收益的、由错误触发的切换，从而在不牺牲性能的前提下控制累计成本。我们推测该差异源自学习信号：Router-R1 主要依靠自然语言路由提示来推断切换何时有帮助，而 MTRouter 直接在轨迹结果（终止得分加退火错误惩罚）上训练，为有效切换提供了更直接的监督。

并非“永不切换”，而是结构化地切换。 人们可能担心上述结果只反映一个极少换模型的保守路由器。图 5 排除了这一可能：MTRouter 在整条 episode 中使用多个模型，但呈现稳定、基准特定的方式，而非将切换作为对错误的条件反射。这表明路由器学习的是一种策略（依赖哪些模型、何时依赖），而不只是通用的“失败就升级”启发式。例如，在 ScienceWorld 上，GPT-5 占早期轮次的 50.8%，而 GPT-OSS 在最后轮次升至 24.3%；相比之下，Router-R1 在各阶段大致各以 ~45% 的比例集中于 DeepSeek 和 Gemini，表现出结构化多样性明显更弱的

模式。

策略的一种具体形式：涌现专长。 为使这种结构化显式化，图 6 测量提升度：模型相对其总体频率在某工具/动作上被过度使用的程度。我们观察到明显、与互补能力一致的专长模式（提升度 > 1 ）：例如在 HLE 中，DeepSeek 在 search 上被过度使用（提升度 1.66），GPT-5 在 python 上为 1.51，Kimi 在 browse 上为 1.98。在 ScienceWorld 上，我们也观察到跨动作类型的类似专长，如 MiniMax 在观察密集型动作上为 1.62，Gemini 在对象交互上为 1.58，GPT-OSS 在查询命令上为 1.81。这些发现联系回主结果：MTRouter 的胜因不是更多切换，而是有选择地切换，并在 episode 过程中为模型分配稳定角色。

学习到的模型嵌入。 图 7 可视化训练后的学习模型嵌入。编码器学会区分候选模型，并按成本层级组织它们，说明它捕获了超越原始属性的、有意义的能力-成本结构。

5 结论

本文提出 MTRouter，一个通过离线学习的结果估计器优化智能体工作流性能-成本权衡的多轮路由框架。通过支持轮级模型选择，我们的方法允许智能体根据不断演化的交互状态自适应地分配计算资源。

在 ScienceWorld 和 HLE 上的实验验证了 MTRouter 的实证有效性。它将 ScienceWorld 平均得分从 48.4 提升至 53.8，同时将总成本降低 58.7%。在 HLE 上，它在保持有竞争力准确率的同时将成本降低 43.4%。除汇总性能外，我们对路由轨迹的分析显示，MTRouter 在不同动作上呈现结构化模型使用，并且比既有基线以更少模型切换获得成功。

局限性

尽管 MTRouter 展现了性能和成本收益，仍存在若干可供未来研究的局限性。

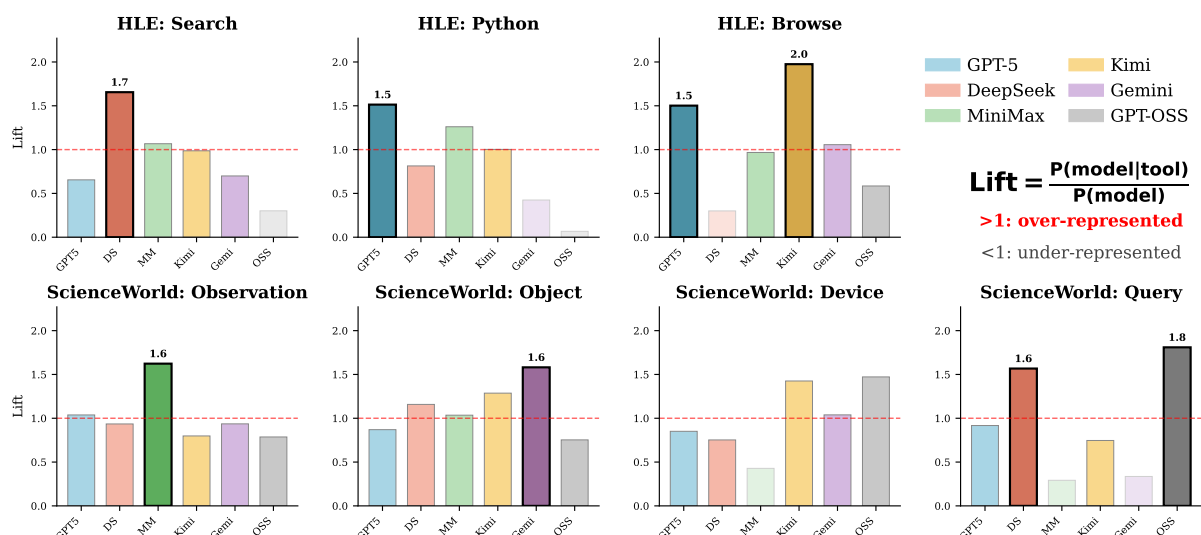


图 6: 按模型划分的工具/动作专长, 以提升度衡量 ($\text{Lift} = P(\text{model} | \text{tool}) / P(\text{model})$)。数值 > 1 表示模型在某工具/动作上占比过高 (专长), 数值 < 1 表示使用不足。

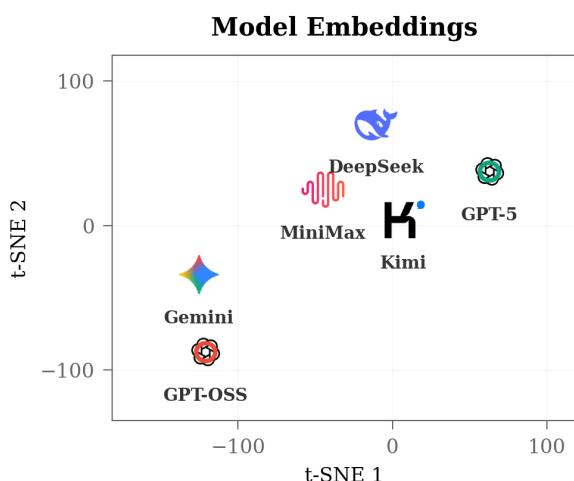


图 7: 模型编码器所学模型嵌入的 t-SNE 可视化。嵌入按身份区分模型, 并形成清晰的成本层级结构; 低成本模型 (如 GPT-OSS、Gemini) 与高成本前沿模型 (如 GPT-5) 分离。

数据收集的可扩展性。 训练轮级路由器的主要瓶颈, 是跨多个候选模型收集多样、长时程轨迹的成本。受计算预算约束, 我们在 ScienceWorld 上评测最多 50 步的 episode, 在 HLE 上评测最多 30 步的 episode。

缺乏在线适应。 目前, MTRouter 运行于离线学习范式。尽管这显著降低训练成本, 路由器却无法实时适应新颖或快速变化的环境。通过强化学习实现在线更新可能进一步提升稳健

性, 但与前沿模型持续在线交互的高昂成本, 使其对当前学术研究仍具挑战。

提示词缓存与切换开销。 多轮路由固有的技术挑战, 是在不同模型之间切换时可能丢失提示词缓存 (例如 KV cache)。在许多基于 API 的部署中, 频繁切换需要新模型重新处理整条轨迹前缀, 从而可能增加延迟和逐轮成本。然而, 我们的分析表明 MTRouter 部分缓解了该问题: 不同于在瞬时错误首次出现时就切换模型的反应式基线, MTRouter 呈现更“从容”的切换模式, 在更长的轮次序列中保持模型稳定性。如成本效率结果所反映, 这种结构化的模型使用有助于平衡自适应模型选择与上下文缓存收益之间的权衡。

致谢

本工作得到国家自然科学基金 (编号 62272092、62172086、62506186) 以及中央高校基本科研业务费专项资金 (编号 N25XQD004) 的支持, 并得到上海人工智能实验室支持。本工作完成于 Yiqun Zhang 和 Hao Li 在上海人工智能实验室实习期间。

References

- Lingjiao Chen, Matei Zaharia, and James Zou. 2023. Frugalpjt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*.
- Shuhao Chen, Weisen Jiang, Baijiong Lin, James Kwok, and Yu Zhang. 2024. Routerdc: Query-based router by dual contrastive learning for assembling large language models. *Advances in Neural Information Processing Systems*, 37:66305–66328.
- Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*.
- Pengfei Gao and Chao Peng. 2025. [More with less: An empirical study of turn-control strategies for efficient coding agents](#). *Preprint*, arXiv:2510.16786.
- Shuyue Hu, Siyue Ren, Yang Chen, Chunjiang Mu, Jinyi Liu, Zhiyao Cui, Yiqun Zhang, Hao Li, Dongzhan Zhou, Jia Xu, and 1 others. 2025. Hands-on llm-based agents: A tutorial for general audiences.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*.
- Logan Kilpatrick and Zach Gleicher. 2025. [Gemini 2.5 Flash-Lite is now stable and generally available](#). Official Google Developers blog. Accessed: 2026-04-18.
- Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, and 1 others. 2025. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*.
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. [Gaia: a benchmark for general ai assistants](#). *Preprint*, arXiv:2311.12983.
- MiniMax. 2025. [Minimax m2 & agent: Ingenious in simplicity](#). Official MiniMax release post. Accessed: 2026-04-18.
- OpenAI, :, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, and 108 others. 2025. [gpt-oss-120b & gpt-oss-20b model card](#). *Preprint*, arXiv:2508.10925.
- OpenAI. 2025. [Introducing GPT-5 for developers](#). Official OpenAI release. Accessed: 2026-04-18.
- OpenRouter. 2025. Openrouter: Unified api for llms and automatic routing. <https://openrouter.ai>. Accessed 2026-01-03.
- Long Phan and et.al. Alice Gatti. 2025. [Humanity’s last exam](#). *Preprint*, arXiv:2501.14249.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551.
- Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, and 1 others. 2025a. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*.
- Tongyi DeepResearch Team, Baixuan Li, Bo Zhang, Dingchu Zhang, Fei Huang, Guangyu Li, Guoxin Chen, Huifeng Yin, Jialong Wu, Jingren Zhou, and 1 others. 2025b. Tongyi deepresearch technical report. *arXiv preprint arXiv:2510.24701*.
- Qianxin Wang, Liqi Chen, Souradip Chakraborty, Qing Dong, Haoran Zhang, Yang Liu, Zhenghai Qi, Tianyi Guo, and Meng Liu. 2024. Embedllm: Learning compact representations of large language models. *arXiv preprint arXiv:2401.11623*.
- Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. 2022. Scienceworld: Is your agent smarter than a 5th grader? *arXiv preprint arXiv:2203.07540*.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, and 1 others. 2024. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094.

John Yang, Carlos E Jimenez, Alexander Wetig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.

Haozhen Zhang, Tao Feng, and Jiaxuan You. 2025a. Router-r1: Teaching llms multi-round routing and aggregation via reinforcement learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Yiqun Zhang, Hao Li, Jianhao Chen, Hangfan Zhang, Peng Ye, Lei Bai, and Shuyue Hu. 2025b. Beyond gpt-5: Making llms cheaper and better via performance-efficiency optimized routing. In *Proceedings of the 2025 7th International Conference on Distributed Artificial Intelligence*, pages 122–129.

Yiqun Zhang, Hao Li, Chenxu Wang, Linyao Chen, Qiaosheng Zhang, Peng Ye, Shi Feng, Daling Wang, Zhen Wang, Xinrun Wang, and 1 others. 2025c. The avengers: A simple recipe for uniting smaller language models to challenge proprietary giants. *arXiv preprint arXiv:2505.19797*.

Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, and 1 others. 2023. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*.

A 数据集与切分细节

A.1 ScienceWorld 任务类型

表 5 列出用于训练和分布外评测的 ScienceWorld 任务类型。

对于每个任务类型，我们最多采样 30 个变体（以限制收集时间）。使用固定随机种子 (42) 将变体按 60%/20%/20% 切分为训练/验证/测试。

切分	任务类型
训练 (13)	boil, melt, chemistry-mix, find-animal, find-plant, grow-plant, identify-life-stages-1, lifespan-longest-lived, inclined-plane-angle, measure-melting-point, power-component, test-conductivity, mendelian-genetics
OOD 测试 (12)	freeze, change-state-of-matter, chemistry-mix-paint, find-non-living, grow-fruit, identify-life-stages-2, lifespan-shortest, inclined-plane-friction, use-thermometer, power-renewable, test-conductivity-unknown, genetics-unknown

表 5: ScienceWorld 任务类型切分。

类别	N (测试)
分布内 (训练)	
数学	200
物理	46
化学	23
生物/医学	37
工程	16
计算机科学/AI	37
分布外	
人文/社会科学	193
其他	176

表 6: 按类别划分的 HLE 测试集分布。

A.2 HLE 学科类别

B 错误检测规则

表 7 给出训练期间用于计算退火错误成本 (AEC) 的错误检测规则完整规范。每条规则包含与环境观察匹配的模式字符串、严重程度 (高/中/低) 和描述。

设计原则。 对于 HLE, 我们区分模型错误 (格式违规、Python 异常) 和外部失败 (HTTP 错误、付费墙)。模型错误因反映可控制的失误而具有更高严重程度; 外部失败依赖第三方服务, 标为低严重程度。对于 ScienceWorld, 我们只惩罚真正无效的动作 (环境解析器无法识别的命令)。“the door is not open” 或 “the object is already in your inventory” 之类的环境反馈属于正常探索, 不被视为错误。

严重程度系数。 在 AEC 中, 严重程度映射到惩罚系数: 高 (1.0)、中 (0.8)、低 (0.2)。warmup 日程使用 $p_0=0.3$ 、 $p_1=0.7$ 、 $w_{\min}=0.3$ 和 $w_{\max}=1.0$ 。这些系数调节基础惩罚 $1/N$, 其中 N 是该任务类别的预期 episode 长度。

C 额外实验

D OpenRouter 基线细节

动机。 OpenRouter (OpenRouter, 2025) 提供自动路由 API, 常用于商用部署。我们将 OpenRouter 作为代表性商用路由基线纳入, 以将 MTRouter 与现成的生产路由系统置于同一语境。

模型池。 不同于将路由限制在固定 6 模型候选池 (表 2) 的设置, OpenRouter 的自动路由可从更广泛的模型集合中选择。在我们的评测中, OpenRouter 基线在下列池中路由 (由 API 在路由时报告):

比较含义。 由于 OpenRouter 可从模型超集 (包括未在我们模型池中出现的多个前沿及提供商特定模型) 中选择, 它代表比我们的设置更强的路由场景。我们仍将其作为实用参考点纳入, 但解释比较时应注意候选模型池不一致。

为何 OpenRouter 在 ScienceWorld 上表现较差? 图 8 展示 OpenRouter 在两个基准上的模型使用分布。在 ScienceWorld 上, OpenRouter 主要选择轻量模型: Mistral-Nemo 占全部模型调用的 68%, 其次是 GPT-5-nano (24%) 和 Claude-4.5 (7%)。这些模型虽然成本高效, 却缺乏 ScienceWorld 的程序化科学任务所需的推理能力。相比之下, 在 HLE 上, OpenRouter 将 54% 的调用分配给更强的前沿模型 Claude-4.5, 同时分配给 Sonar (23%) 和 Mistral-Nemo (15%), 反映了对该基准更恰当的难度评估。

这一差异揭示了通用商用路由器的一个关键局限: 若没有任务特定训练, 它们可能低估陌生领域难度, 并过度依赖更便宜的模型。Sci-

enceWorld 的文本接口和看似简单的命令可能使路由器误将其视为简单任务, 但它实际上需要多步规划和精确动作序列, 较弱模型难以执行。环境评分函数给出的负分 (测试为 -26.4 , OOD 为 -26.9) 表明, 所选模型常无法朝任务完成取得有意义的进展。

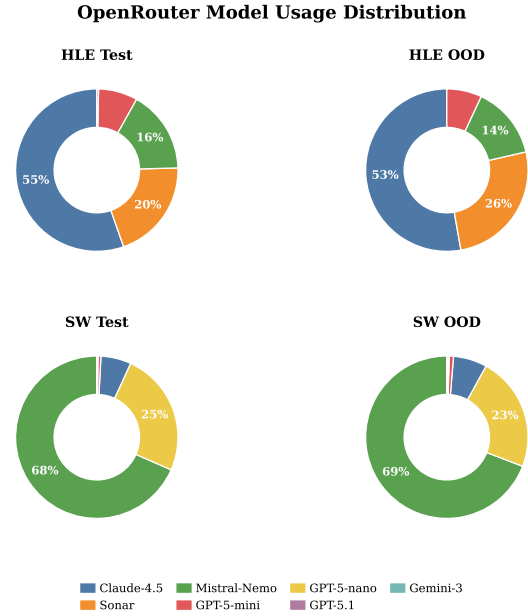


图 8: OpenRouter 在 ScienceWorld 和 HLE 上的模型使用分布。在 ScienceWorld 上, OpenRouter 主要使用轻量模型 (Mistral-Nemo: 68%, GPT-5-nano: 24%); 在 HLE 上, 它主要使用更强模型 (Claude-4.5: 54%), 反映出不同的难度评估。

E 提示词与工具 schema

我们报告评测设置中使用的提示词。为节省篇幅, 省略格式错误纠正提示词。

E.1 ScienceWorld 智能体提示词

E.2 HLE 智能体提示词

HLE 系统提示词通过模板变量注入工具 schema。我们在附录 E.3 中逐一列出每个工具 schema。

E.3 工具 schema

我们列出 HLE 智能体使用的各工具 JSON schema (名称、描述与参数 schema)。

类别	规则名	严重程度	描述
<i>HLE 基准</i>			
格式错误	format_error	中	工具调用格式错误—模型未遵循要求格式
	tool_invalid_args	中	无效工具参数或缺少必需参数
	tool_parse_error	中	工具调用解析失败
	tool_unknown	高	模型调用了不存在的工具名
Python 执行 错误	python_traceback	高	带 traceback 的 Python 执行异常
	python_name_error	高	引用了未定义变量
	python_syntax_error	高	Python 语法错误
	python_indentation_error	高	Python 缩进错误
	python_type_error	高	类型不匹配错误
	python_value_error	中	无效值错误
	python_index_error	中	索引越界
	python_key_error	中	未找到字典键
	python_attribute_error	中	属性访问错误
	python_import_error	中	模块导入失败
	python_zero_division	中	除零
	python_timeout	高	代码执行超时
搜索工具 错误	search_no_results	高	搜索未返回结果
	search_http_error	低	搜索期间 HTTP 连接错误
	search_rate_limit	低	超出搜索速率限制
浏览工具 错误	browse_403	低	HTTP 403 禁止访问
	browse_404	低	HTTP 404 未找到
	browse_access_denied	低	资源访问被拒绝
	browse_paywall	低	付费墙或订阅拦截
	browse_timeout	低	浏览请求超时
<i>Science World 基准</i>			
无效动作	no_known_action	高	环境解析器无法识别的无效动作命令

表 7: 用于退火错误成本 (AEC) 计算的详细错误检测规则。严重程度决定惩罚系数: 高 (1.0)、中 (0.8)、低 (0.2)。浏览工具错误标为低严重程度, 因为它们反映外部失败而非模型错误。

ScienceWorld 系统提示词

你是一名与文本式科学模拟环境交互的有帮助助手。你的目标是通过发出文本命令完成科学实验。

交互方式

请将命令写在 “text 代码块中。环境将以观察结果回应。

<format_example>

THOUGHT: 我应该先探索周围环境。

“text

look around

““

</format_example>

可用命令类型

常见命令包括：

- 移动：go to [location]、open door、go through door
- 交互：pick up [object]、put [object] in [container]、activate [object]
- 观察：look around、look at [object]、inventory
- 任务特定：focus on [object]、use [tool] on [object]

查询命令（免费动作）

你可在不消耗游戏轮次的情况下查询可用动作：

- ?navigation：显示移动动作（go、walk、move）
- ?object：显示对象操作（pick up、put、pour）
- ?observation：显示观察动作（look、examine、inventory）
- ?device：显示设备控制（activate、turn on/off、use）
- ?door：显示门/容器动作（open、close）
- ?electrical：显示电气动作（connect、disconnect）
- ?interaction：显示交互动作（mix、eat、focus）
- ?all：显示所有有效动作
- ?categories：显示查询帮助

在决定下一步之前，使用查询探索可用动作。

重要说明

- 每次响应只发出一条命令
- 包含解释推理的 THOUGHT 部分
- 环境按轮次交互—在发出下一条命令前等待观察结果
- 部分任务需要多步才能完成

ScienceWorld 实例提示词

<task_description>

{{task_description}}

</task_description>

<initial_observation>

{{initial_observation}}

</initial_observation>

<instructions>

完成上文描述的科学实验。你正在与模拟环境交互。请一次发出一条命令并观察结果。

提示：使用 ?navigation 或 ?object 等查询命令探索可用动作，且不消耗轮次。

策略提示：

- 先探索：使用 open door to [room]，再使用 go to [room] 导航
- 观察每个房间，寻找所需对象
- 使用 pick up [object] 拾取对象
- 加热时：找到炉灶，用 activate [stove] 打开，将容器放到其上
- 冷却时：使用冷冻柜或冰箱
- 使用 focus on [object] 检查物质

为完成任务，请执行任务描述所需的动作。当你认为任务完成时，发出命令：

“text

task completed

““

请记住：

- 逐步思考需要哪些动作
- 探索环境以找到所需对象
- 有些动作可能需要前提条件（例如先拾取对象才能使用它）

</instructions>

HLE 系统提示词

你是一名专家问题求解者，正在解决 Humanity's Last Exam 中的挑战性问题。这些问题需要深入推理、谨慎研究和精确答案。

可用工具

```
<tools>
{% for tool in tool_schemas %}
{{ tool.to_xml() }}
{% endfor %}
</tools>
```

工具调用格式

你可以在 `<tool_call>` 标签中使用 XML 或 JSON 格式：

格式 A：XML

```
<tool_call>
<tool_name>
<param1>value1</param1>
<param2>value2</param2>
</tool_name>
</tool_call>
```

格式 B：JSON

```
<tool_call>{"name": "tool_name", "arguments": {"param1": "value1"}}</tool_call>
```

示例

```
<tool_call>
<search>
<query>maximum likelihood estimation</query>
</search>
</tool_call>

<tool_call>{"name": "python", "arguments": {"code": "print(2 + 2)"}}</tool_call>
```

HLE 问题策略

1. **理解问题：**仔细阅读并识别所问内容
2. **拆分问题：**必要时将其分解为子问题
3. **必要时研究：**对不确定的事实信息使用 `search`
4. **核验来源：**准确性重要时，使用 `browse` 阅读一手来源
5. **必要时计算：**使用 `python` 进行计算和数据分析
6. **综合：**整合多个来源的信息
7. **核验答案：**提交前再次检查
8. **提交：**使用 `answer` 提交最终答案

重要说明

- 这些是挑战性问题—请从容作答
- 务必精确—通常要求精确答案
- 使用工具前展示你的推理
- 若不确定，请在答案中表达置信度

HLE 实例提示词

HLE 问题

{% if subject %}

学科: {{ subject }}

{% endif %}

{{ question | default(task) }}

请逐步求解。使用工具进行研究与核验。使用 `answer` 工具提交最终答案:

```
<tool_call>{"name": "answer", "arguments": {"answer": "your final answer"}}</tool_call>
```

工具 Schema: search

```
{
  "name": "search",
  "description": "Search the web for information using Google",
  "parameters": {
    "type": "object",
    "properties": {
      "query": {
        "oneOf": [
          { "type": "string" },
          { "type": "array", "items": { "type": "string" } }
        ],
        "description": "Search query or list of queries for batch search"
      },
      "num_results": {
        "type": "integer",
        "description": "Number of results to return (default: 10)",
        "minimum": 1,
        "maximum": 100
      }
    }
  },
  "required": [ "query" ]
}
```

工具 Schema: browse

```
{
  "name": "browse",
  "description": "Visit webpage(s) and extract information based on a goal",
  "parameters": {
    "type": "object",
    "properties": {
      "url": {
        "type": "string",
        "description": "URL to visit. Can be a single URL or array of URLs."
      },
      "goal": {
        "type": "string",
        "description": "The goal of the visit - what information to extract from the webpage(s)."
      },
      "extract_mode": {
        "type": "string",
        "enum": [ "text", "markdown", "html" ],
        "description": "Content extraction mode (default: text). Only used when LLM summary is disabled."
      }
    }
  },
  "required": [ "url", "goal" ]
}
```

工具 Schema: python

```
{
  "name": "python",
  "description": "Execute Python code and return the output",
  "parameters": {
    "type": "object",
    "properties": {
      "code": {
        "type": "string",
        "description": "Python code to execute"
      }
    }
  },
  "required": [ "code" ]
}
```

工具 Schema: answer

```
{
  "name": "answer",
  "description": "Submit your final answer to complete the task",
  "parameters": {
    "type": "object",
    "properties": {
      "answer": {
        "type": "string",
        "description": "The final answer to submit"
      },
      "confidence": {
        "type": "number",
        "description": "Confidence level from 0 to 1 (optional)",
        "minimum": 0,
        "maximum": 1
      },
      "reasoning": {
        "type": "string",
        "description": "Brief explanation of how you arrived at the answer (optional)"
      }
    }
  },
  "required": [ "answer" ]
}
```

基准	预算	得分/准确率 $\uparrow$	总成本 (\$) $\downarrow$
ScienceWorld	$0.5 \times B$	45.2 ± 3.8	3.6
ScienceWorld	$1.0 \times B$	53.8 ± 3.2	5.7
ScienceWorld	$2.0 \times B$	55.6 ± 3.1	8.1
HLE	$0.5 \times B$	20.6 ± 2.8	23.4
HLE	$1.0 \times B$	26.0 ± 2.3	35.0
HLE	$2.0 \times B$	27.3 ± 2.2	44.8

表 8: 预算敏感性。放宽预算会提升性能，但存在明显的边际收益递减。

池	ScienceWorld 测试		ScienceWorld OOD		HLE 测试		HLE OOD	
	得分 $\uparrow$	成本 $\downarrow$	得分 $\uparrow$	成本 $\downarrow$	准确率 $\uparrow$	成本 $\downarrow$	准确率 $\uparrow$	成本 $\downarrow$
2 模型	50.6 ± 3.6	8.9	7.2 ± 4.3	28.5	25.4 ± 2.5	45.2	36.4 ± 3.1	43.5
6 模型	53.8 ± 3.2	5.7	9.9 ± 3.9	16.3	26.0 ± 2.3	35.0	38.6 ± 3.0	31.2
8 模型	54.1 ± 3.3	5.3	10.3 ± 4.1	16.8	25.8 ± 2.4	34.2	39.1 ± 2.9	29.5

表 9: 在 2、6、8 模型设置下的候选池敏感性。

历史预算	ScienceWorld $\uparrow$	HLE $\uparrow$
max_tokens=2048	49.3 ± 3.7	23.8 ± 2.6
max_tokens=4096	52.1 ± 3.4	25.3 ± 2.4
max_tokens=8192	53.8 ± 3.2	26.0 ± 2.3
max_tokens=16384	53.5 ± 3.3	26.2 ± 2.3

表 10: 对历史 token 预算的敏感性。

E.4 用于 LLM Router / Router-R1 的路由模型提示词

我们报告 LLM Router 基线和 Router-R1 使用的轮级路由提示词。两个基线在每一轮查询路由模型来产生一个 `<select>` 决策，并使用相同的提示词模板；Router-R1 使用训练后的 Qwen2.5-7B-Instruct 路由模型，而 LLM Router 直接使用 DeepSeek-V3.2（不训练）。

E.5 浏览提取器提示词

HLE 的 browse 工具可选择使用 LLM，从检索到的网页中提取和摘要相关证据。我们报告该基于 LLM 摘要的提取器提示词。

E.6 HLE 裁判提示词

HLE 评分使用与官方 HLE 评测对齐的 LLM-as-a-judge 组件。我们报告用于将模型响应与给定参考答案比较的裁判提示词。

LLM Router / Router-R1 系统提示词（策略 LLM）

你是一名模型路由助手。你的工作是为给定任务选择最佳模型。

可用模型：

{model_list}

说明

1. 在 `<think>...</think>` 标签中分析任务
2. 使用 `<select>model_name</select>` 选择最佳模型

示例

`<think>` 这是一个简单的数学问题。更便宜的模型已足够。`</think>`
`<select>deepseek/deepseek-v3.2</select>`

规则

- 你必须且只能输出一个 `<select>` 标签
- 模型名称必须与可用列表完全一致
- 考虑任务复杂度、模型优势和成本效益

LLM Router / Router-R1 模型描述

openai/gpt-5: 成本 1.25/10, 上下文 400K。SW 成功率 48%、HLE 成功率 25%，表现最佳。适用于复杂多步推理和困难科学问题。

openai/gpt-oss-120b: 成本 0.09/0.36, 上下文 131K。SW 成功率 33%、HLE 成功率 10%，在中等难度任务上成本效益极佳。

moonshotai/kimi-k2-0905: 成本 0.39/1.9, 上下文 131K。SW 成功率 31%、HLE 成功率 11%。适合通用任务，指令遵循良好。

deepseek/deepseek-v3.2: 成本 0.27/0.42, 上下文 164K。SW 成功率 16%、HLE 成功率 16%。擅长数学和编码，适用于结构化推理任务。

google/gemini-2.5-flash-lite: 成本 0.1/0.4, 上下文 1M。SW 成功率 23%、HLE 成功率 6%。上下文窗口最大，适用于长文档分析。

minimax/minimax-m2: 成本 0.2/1, 上下文 196K。SW 成功率 22%、HLE 成功率 8%。推理快速，适用于简单问答和直接任务。

提供商/模型	提供商/模型
openai/gpt-5.1	openai/gpt-5
openai/gpt-5-mini	openai/gpt-5-nano
openai/gpt-4.1	openai/gpt-4.1-mini
openai/gpt-4.1-nano	openai/gpt-4o
openai/gpt-4o-2024-05-13	openai/gpt-4o-2024-08-06
openai/gpt-4o-2024-11-20	openai/gpt-4o-mini
openai/gpt-4o-mini-2024-07-18	openai/gpt-4-turbo
openai/gpt-4-turbo-preview	openai/gpt-4-1106-preview
openai/gpt-4	openai/gpt-3.5-turbo
openai/gpt-oss-120b	anthropic/claude-opus-4.5
anthropic/claude-opus-4.1	anthropic/claude-opus-4
anthropic/claude-sonnet-4.5	anthropic/claude-sonnet-4
anthropic/claude-3.7-sonnet	anthropic/claude-haiku-4.5
anthropic/claude-3.5-haiku	anthropic/claude-3-haiku
google/gemini-3-pro-preview	google/gemini-2.5-pro
google/gemini-2.0-flash-001	google/gemini-2.5-flash
mistralai/mistral-large	mistralai/mistral-large-2407
mistralai/mistral-large-2411	mistralai/mistral-medium-3.1
mistralai/mistral-nemo	mistralai/mistral-7b-instruct
mistralai/mixtral-8x7b-instruct	mistralai/mixtral-8x22b-instruct
mistralai/codestral-2508	x-ai/grok-4
x-ai/grok-3	x-ai/grok-3-mini
deepseek/deepseek-r1	meta-llama/llama-3.3-70b-instruct
meta-llama/llama-3.1-405b-instruct	meta-llama/llama-3.1-70b-instruct
meta-llama/llama-3.1-8b-instruct	meta-llama/llama-3-70b-instruct
meta-llama/llama-3-8b-instruct	qwen/qwen3-235b-a22b
qwen/qwen3-32b	qwen/qwen3-14b
cohere/command-r-plus-08-2024	cohere/command-r-08-2024
moonshotai/kimi-k2-thinking	perplexity/sonar

表 11: 我们的 OpenRouter 基线使用的 OpenRouter 自动路由模型池（由 API 在路由时报告）。

浏览提取器提示词 (LLM 摘要)

请处理以下网页内容和用户目标，以提取相关信息：

网页内容

{webpage__content}

用户目标

{goal}

任务指南

1. **为推理扫描内容：**在网页内容中定位与用户目标直接相关的**具体章节/数据**。
2. **为证据提取关键信息：**从内容中识别并提取**最相关的信息**；不要遗漏重要信息。尽可能输出**完整的原始上下文**（可超过三段）。
3. **为摘要组织输出：**组织为逻辑流畅的简洁段落，优先保证清晰度，并判断信息对目标的贡献。

输出格式：包含 "rational"、"evidence" 和 "summary" 字段的 JSON 格式。

HLE 裁判提示词

根据下方精确且无歧义的 [correct_answer]，判断以下对 [question] 的 [response] 是否正确。

[question]: {question}

[response]: {response}

你的判定必须符合以下格式和标准：

extracted_final_answer: 从 [response] 提取的最终精确答案。若响应中没有可提取的精确最终答案，将提取结果设为 None。

[correct_answer]: {correct_answer}

reasoning: 根据 [correct_answer] 解释 extracted_final_answer 为何正确或错误，只关注是否存在实质性差异。不要评论背景，不要尝试解决问题，不要为不同于 [correct_answer] 的答案辩护；只关注答案是否匹配。

correct: 若 extracted_final_answer 与上方给定的 [correct_answer] 匹配，或数值问题的误差在小范围内，回答 yes；否则回答 no。

confidence: 从 [response] 提取的 0% 到 100% 置信度分数。若没有置信度分数可用，填 100。

以包含四个字段的 JSON 对象作答：extracted_final_answer、reasoning、correct、confidence。